

TMS320C5515 上板验证完整指南

目标：把当前 PC 上跑通的 C 代码移到 TMS320C5515 开发板上，用真实音频信号验证算法效果。

目录

- 0. [必备清单](#)
- 1. [CCS 安装](#)
- 2. [获取并安装 CSL / BSL](#)
- 3. [创建 CCS 工程](#)
- 4. [导入源码](#)
- 5. [替换 fft_hwa.c \(用 HWA\)](#)
- 6. [实现 app_main.c 里的 BSL/CSL 桩](#)
- 7. [配置 Linker .cmd](#)
- 8. [连接开发板](#)
- 9. [编译 + 加载到板](#)
- 10. [实时音频验证](#)
- 11. [真实 Cycle 数测量](#)

0. 必备清单

0.1 硬件

项目	推荐型号	说明
开发板	TI TMDX5515EZDSP (eZdsp USB Stick) 或 C5515 EVM	板上含 C5515 + AIC3204 codec + 3.5mm 输入输出 + USB JTAG (XDS100v2)
JTAG 仿真器	板上集成 XDS100v2 (USB)	不需要单买仿真器
USB 线	Mini-USB 或 Micro-USB (看具体板子)	同时供电+JTAG
音频线	3.5mm 公-公 ×2	一根接信号源，一根接耳机/喇叭
信号源	手机播放 build/synth.wav 或语音	也可用函数发生器
输出	耳机或有源喇叭	不要用大功率被动喇叭直推

0.2 主机（运行 CCS 的电脑）

配置	最低	推荐
OS	Windows 10/11 x86_64 或 Ubuntu 20.04+ x86_64	Windows 10/11
RAM	4 GB	8-16 GB
硬盘	5 GB 空闲	10 GB 空闲
USB	USB 2.0	USB 2.0

1. CCS 安装

1.1 下载安装器

1. 打开 <https://www.ti.com/tool/CCSTUDIO>
2. 点击 Download options
3. 选择 Offline installer (约 3.5 GB, 含所有设备包)。不要选 Web installer, 网络中断很麻烦。
4. 选你的 OS (Windows 64-bit 或 Linux 64-bit)
5. 登录 TI 账号后会发邮件给下载链接

1.2 Windows 安装步骤

1. 双击 ccs_setup_12.x.x.exe
 - 如果触发 SmartScreen 警告: "More info" → "Run anyway" (TI 没买证书签名)
2. License agreement → I accept → Next
3. Installation directory: 默认 C:\ti\ccs1210\
 - 路径不能有中文或空格, 比如 C:\Program Files\ 不行
4. Setup Type → 选 Custom Installation
5. Component selection — 关键页面! 只勾这些:
 - C5000 ultra-low-power DSP ← C5515 在这里
 - C2000、C6000、Sitara、SimpleLink、MSP430 等其它族 — 全部取消
 - GUI Composer Runtime (默认勾上, 保留)
6. Debug Probes 页:
 - TI XDS Debug Probe Support (板上的 XDS100v2 用这个驱动)
 - 其它都取消
7. Next → 等待下载和安装 (30-45 分钟)
8. 安装结束后桌面会有 "Code Composer Studio 12" 快捷方式

1.3 Linux 安装步骤

```
# 解压
tar -xzf CCS12.x.x.x_linux-x64.tar.gz
cd CCS12.x.x.x_linux-x64

# 必装的系统库
sudo apt-get install libc6-i386 libusb-1.0-0 libgconf-2-4 libcanberra-gtk-module

# 启动安装器 (GUI)
./ccs_setup_12.x.x.run

# 后续步骤同 windows 1.2 的 4-7 步
```

安装完后：

```
# udev 规则（让普通用户能访问 USB JTAG）
sudo cp ~/ti/ccs1210/ccs/install_scripts/install_drivers.sh /tmp/
sudo bash /tmp/install_drivers.sh
sudo udevadm control --reload-rules
```

1.4 首次启动 CCS

1. 启动 CCS
2. Workspace 选一个路径，比如 C:\projects\ccs_workspace（同样不要中文/空格）
3. 第一次启动会显示 "TI Resource Explorer"，可以关掉
4. Help → About 确认版本 12.x，Window → Show View → Other → C/C++ → Project Explorer 打开工程视图

2. 获取并安装 CSL / BSL

2.1 下载

1. C55x CSL v3.08（约 30 MB）：
 - 链接：<https://www.ti.com/lit/zip/sprc133>
 - 解压到 C:\ti\c55_csl_3.08\
2. C5515 BSL（约 5 MB）：
 - 链接：<https://www.ti.com/lit/zip/sprc580>
 - 解压到 C:\ti\c5515_bsl_1.10\

确保以下目录存在：

```
C:\ti\c55_csl_3.08\
├─ inc\                ← CSL 头文件 (csl_fft.h, csl_dma.h, ...)
├─ lib\
│   └─ csl_c5515.lib    ← CSL 库
└─ examples\

C:\ti\c5515_bsl_1.10\
├─ inc\
├─ lib\
│   └─ 5515_bsl.lib
```

如果 lib 文件不存在，需要先打开 CSL 工程编译生成 — 见 CSL 包内的 README。

2.3 拷贝 HWAFFT 库

C5515 的 FFT 协处理器有专门的库（不在标准 CSL 里）：

1. 从 TI 网站下载 HWAFFT 库（sprc782 或类似）
2. 解压，把 hwafft.lib 拷到 C:\ti\c55_csl_3.08\lib\
3. 把 hwafft.h 拷到 C:\ti\c55_csl_3.08\inc\

或者直接使用 CSL 自带的 FFT 接口（如果版本支持）。

3. 创建 CCS 工程

3.1 拷贝项目源码到 Windows / x86_64 Linux

把整个 tms320/ 目录从开发机拷到 CCS 主机：

```
# Linux 开发机上：
tar -czf tms320.tar.gz tms320/

# 然后用 U 盘 / scp / 共享文件夹 拷到 CCS 主机
# 放到 C:\projects\tms320\ (Windows)
# 或 ~/projects/tms320/ (Linux)
```

3.2 在 CCS 创建空工程

1. File → New → CCS Project
2. Target 区域：
 - Family: C5500 Ultra Low Power DSP
 - Variant: C55XX → 下拉选 TMS320C5515
 - Connection: Texas Instruments XDS100v2 USB Debug Probe
3. Project name: hearing_aid
4. Use default location: 取消勾选，手动设到 C:\projects\tms320\ccs\hearing_aid\
5. Project templates and examples → 展开 → 选 Empty Projects → Empty Project
6. Finish

工程创建后，Project Explorer 里会看到 hearing_aid 节点。

3.3 调整工程编译选项

右键工程 → Properties → 打开属性页：

3.3.1 C 标准

C/C++ Build → Settings → Tool Settings → C5500 Compiler → Advanced Options → Language Options:

- C dialect: Relaxed ISO (C99 兼容；C5500 编译器对纯 C99 支持有限)

3.3.2 优化等级

C/C++ Build → Settings → Tool Settings → C5500 Compiler → Optimization:

- Optimization level: -o2 (函数级 + 软件流水)

3.3.3 预处理宏定义

C/C++ Build → Settings → Tool Settings → C5500 Compiler → Predefined Symbols:

- 点 + 加: TMS320_C5515
- 点 + 加: _C55XX (CSL 库内部用)

3.3.4 Include 路径

C/C++ Build → Settings → Tool Settings → C5500 Compiler → Include Options → Add dir to #include search path:

- 加: \${PROJECT_ROOT}/src
- 加: C:\ti\c55_csl_3.08\inc

- 加: C:\ti\c5515_bsl_1.10\inc

3.3.5 链接器库路径和库

C/C++ Build → Settings → Tool Settings → C5500 Linker → File Search Path:

- Add
to library search path: C:\ti\c55_csl_3.08\lib, C:\ti\c5515_bsl_1.10\lib
- Include library file or command file as input:
 - csl_c5515.lib
 - 5515_bsl.lib
 - hwafft.lib (如果用了 HWAAFFT)
 - rts55.lib (默认 C 运行时; 通常已自动加上)

3.4 应用并关闭属性页

点 Apply and Close。

4. 导入源码

4.1 拷贝 vs 链接

CCS 有两种方式把外部文件加入工程:

- Copy files: 把文件复制进工程目录, 独立维护
- Link to files: 在工程里建符号链接, 文件保留在原位置

推荐: Link, 这样原始代码和 CCS 工程能同步修改。

4.2 添加源文件

1. 右键工程 → Add Files...

2. 浏览到 C:\projects\tms320\src\

3. 多选 (按住 Ctrl) 以下文件:

必加 (algo 层 + util) :

- src/config.h
- src/algo/*.c, *.h (全部 5 对: pipeline、stft、wiener、vad_decision、transpose)
- src/util/fixed_point.h
- src/util/window_table.c, .h
- src/util/twiddle_table.c, .h

必加 (HAL 层) :

- src/hal/cycle_counter.c, .h
- src/hal/memmap.h
- 暂时也加 src/hal/fft_hwa.h (保留接口)
- src/hal/fft_hwa.c ← 稍后第 5 章会替换内容

必加 (板上 entry) :

- src/app_main.c

不要加 (PC-only) :

- src/main_pc.c
- src/util/wav_io.c, .h

- test/* 所有文件
 - tools/* 所有文件
4. 弹出 Copy / Link 对话框 → 选 Link to files → OK

4.3 排除 main_pc.c (如果误加了)

如果不小心加了 main_pc.c:

- 右键文件 → Resource Configurations → Exclude from Build → Select All → OK
- 这样它不参与编译, 但还在工程里方便对照

5. 替换 fft_hwa.c (用 HWA)

唯一需要改的算法/HAL 文件。原来的 fft_hwa.c 是软件 FFT; 上板要换成调 HWA 协处理器。

5.1 备份原文件

```
src/hal/fft_hwa.c → src/hal/fft_hwa_sw.c.bak    (改名, 保留软件版本作对照)
```

或在 CCS 里右键 → Exclude from Build。

5.2 写新的 fft_hwa.c

创建 src/hal/fft_hwa.c (内容如下), 覆盖原文件:

```
/* =====
 * fft_hwa.c  --  TMS320C5515 HWA FFT, on-chip CSL implementation
 *
 * Replaces the software radix-2 fft_hwa.c used on PC.  Calls the
 * on-chip FFT hardware accelerator via the HWAFFT driver.  The
 * input/output Q15 complex format and the per-stage scaling match
 * what the PC software version produced, so no upstream algo code
 * needs to change.
 * ===== */

#include "fft_hwa.h"
#include "cycle_counter.h"
#include "memmap.h"
#include <hwafft.h>      /* or <cs1_fft.h> depending on CSL version */
#include <string.h>

static uint32_t g_last_cycles = 0;

/* HWAFFT scratch buffers must live in DARAM (HWA uses ping-pong DMA).
 * The format is interleaved Q15: [Re0,Im0,Re1,Im1,...,Im(N-1)]. */
#pragma DATA_SECTION(g_fft_data, ".daram");
#pragma DATA_SECTION(g_fft_scratch, ".daram");
#pragma DATA_ALIGN (g_fft_data, 4);
#pragma DATA_ALIGN (g_fft_scratch, 4);
static int16_t g_fft_data [N_FFT * 2];
static int16_t g_fft_scratch[N_FFT * 2];

void fft_hwa_init(void) {
    /* HWAFFT is a stateless library; no per-instance state.  But the
     * IRAM permission bit must be enabled. */
    hwafft_iram_setup();    /* check exact name in your hwafft.h */
}
```

```

}

void fft_hwa_forward(const q15_t time_in[N_FFT],
                    q15_cplx_t freq_out[N_FFT]) {
    int i;

    /* 1. Pack real input as interleaved Re,Im with Im=0. */
    for (i = 0; i < N_FFT; ++i) {
        g_fft_data[2*i] = time_in[i];
        g_fft_data[2*i+1] = 0;
    }

    /* 2. Run the 1024-pt forward FFT with per-stage scaling enabled. */
    hwafft_1024pts(g_fft_data, g_fft_scratch,
                  HWAFFT_FORWARD_FFT,
                  HWAFFT_SCALE);    /* names vary by CSL version */

    /* 3. Unpack to q15_cplx_t output. */
    for (i = 0; i < N_FFT; ++i) {
        freq_out[i].re = g_fft_data[2*i];
        freq_out[i].im = g_fft_data[2*i+1];
    }

    g_last_cycles = CYC_FFT_HWA_1024;
    cycles_add(CYC_FFT_HWA_1024);
}

void fft_hwa_inverse(const q15_cplx_t freq_in[N_FFT],
                    q15_t time_out[N_FFT]) {
    int i;

    for (i = 0; i < N_FFT; ++i) {
        g_fft_data[2*i] = freq_in[i].re;
        g_fft_data[2*i+1] = freq_in[i].im;
    }

    hwafft_1024pts(g_fft_data, g_fft_scratch,
                  HWAFFT_INVERSE_FFT,
                  HWAFFT_NOSCALE);

    /* Take real part only. */
    for (i = 0; i < N_FFT; ++i) {
        time_out[i] = g_fft_data[2*i];
    }

    g_last_cycles = CYC_FFT_HWA_1024;
    cycles_add(CYC_FFT_HWA_1024);
}

uint32_t fft_hwa_last_cycles(void) { return g_last_cycles; }

```

5.3 找你版本 CSL 里的实际函数名

不同 CSL 版本里函数名略有差异。要查的：

- 1024 点 FFT 函数（可能是 hwafft_1024pts、HWAFFT_run1024Pts、FFT_1024）
- Forward / Inverse 标志名
- Scale enable / disable 标志名

最快办法：

1. 打开 C:\ti\c55_csl_3.08\inc\hwafft.h（或 csl_fft.h）
2. 搜 "1024"
3. 看头文件里实际声明的函数原型和宏

最权威参考是 CSL 自带的 examples：

- C:\ti\c55_csl_3.08\examples\hwafft\ 下的工程
- 它给出了 HWA FFT 的完整 init + 调用样例

5.4 验证 fft_hwa 替换正确

加入工程后：

- Project → Build Project
- 应该 0 编译错误
- 如果报 "undefined symbol hwafft_1024pts"：检查链接器有没有加 hwafft.lib（§3.3.5）

6. 实现 app_main.c 里的 BSL/CSL 桩

src/app_main.c 当前的 4 个 extern 是占位符。要换成真实 CSL/BSL 调用。

6.1 找参考代码

最佳参考：TI eZdsp 板套件 examples 里的 AIC3204 loopback 示例。它做的事就是：

- 初始化 AIC3204 codec
- 启动 DMA ping-pong I/O
- ISR 里把输入直接拷给输出

我们只需要把 ISR 里的"直接拷贝"换成 pipeline_process(in, out) 就行。

下载入口：

- TI 官方：<https://www.ti.com/tool/SPRC580> 里的 examples
- 或 eZdsp 板套件 CD \examples\sample_audio\
- 或 GitHub 搜 "TMS320C5515 aic3204 loopback"

6.2 改写 app_main.c

打开 src/app_main.c，把 #ifdef TMS320_C5515 里面的内容整体替换为：

```
#ifdef TMS320_C5515

#include "config.h"
#include "util/fixed_point.h"
#include "algo/pipeline.h"
#include "hal/memmap.h"

#include <csl_general.h>
```



```

#include <csl_audioClass.h>
#include <csl_dma.h>
#include <csl_intc.h>
#include <csl_pll.h>
#include <stdint.h>

/* ---- Ping-pong audio buffers (DMA-accessible DARAM) ----- */
#pragma DATA_SECTION(g_dma_in, ".daram");
#pragma DATA_SECTION(g_dma_out, ".daram");
#pragma DATA_ALIGN (g_dma_in, 4);
#pragma DATA_ALIGN (g_dma_out, 4);
static int16_t g_dma_in [HOP * 2]; /* ping-pong = 2 halves */
static int16_t g_dma_out[HOP * 2];
static volatile int g_active_half = 0;

/* ---- Codec init ----
 * Sets up:
 *   PLL -> 100 MHz core clock
 *   AIC3204 codec via I2C: 16 kHz, mono, 16-bit, line-in/line-out
 *   McBSP1 in I2S mode for audio data */
static void audio_codec_init(uint32_t sample_rate) {
    CSL_PLL_CONFIG pllCfg = { /* 100 MHz from 12 MHz xtal */
        0x8000, 0x0000, 0x0806, 0x0000 /* CONTROL1..4, check datasheet */
    };
    CSL_PLL_init(&pllCfg);

    CSL_AudioClassHandle hAudio;
    CSL_AudioClassObj    audioObj;
    CSL_Status           status;

    hAudio = CSL_audioClassOpen(&audioObj, CSL_AUDIO_CLASS_0, NULL, &status);
    if (status != CSL_SOK) for(;;); /* hang on error */

    CSL_AudioClassConfig cfg;
    cfg.sampleRate = sample_rate;
    cfg.dataLength = 16;
    cfg.numChannels = 1;
    cfg.opMode      = CSL_AUDIO_CLASS_OPMODE_DMA_INTERRUPT;
    CSL_audioClassConfig(hAudio, &cfg);
}

/* ---- DMA start ----
 * Configure DMA0 (channel pair) to do ping-pong I/O:
 *   ch0: McBSP1 RX -> g_dma_in
 *   ch1: g_dma_out -> McBSP1 TX
 * Each half-buffer completion fires an IRQ. */
static void dma_audio_start(void) {
    /* Boilerplate ~50 lines. See AIC3204 loopback example for full setup. */
    /* Key registers: DMACH0SSAL/SAH/DAL/DAH/SIZL/SIZH/CTRL */
    /* ... copied verbatim from CSL example ... */
}

/* ---- ISR ----
 * Invoked by DMA "half-buffer complete" IRQ. Runs in ISR context;
 * keep it short (<10K cycles to stay well under 16 ms budget). */
interrupt void audio_isr(void) { /* note: "interrupt" keyword for C55 */
    const q15_t *in_hop = (const q15_t *)&g_dma_in [g_active_half * HOP];
    q15_t *out_hop = (q15_t *)&g_dma_out[g_active_half * HOP];

```

```

    pipeline_process(in_hop, out_hop);
    g_active_half ^= 1;

    /* Acknowledge DMA interrupt */
    CSL_intcEvtClear(CSL_INTC_EVTID_DMA);
}

/* ---- main ---- */
int main(void) {
    /* 1. Disable all interrupts during setup. */
    CSL_intcGlobalDisable();

    /* 2. Init the pipeline (FFT, Wiener, VAD, etc.) */
    pipeline_init();

    /* 3. Init codec + DMA */
    audio_codec_init(FS);
    dma_audio_start();

    /* 4. Hook our ISR to the DMA vector */
    CSL_intcHookIsr(CSL_INTC_EVTID_DMA, (CSL_IsrPtr)audio_isr);

    /* 5. Enable DMA IRQ + global IRQs */
    CSL_intcEvtEnable(CSL_INTC_EVTID_DMA);
    CSL_intcGlobalEnable();

    /* 6. Idle loop -- all work happens in audio_isr */
    for (;;) {
        asm(" idle");    /* C55 IDLE instruction, low-power wait for IRQ */
    }
}

#endif /* TMS320_C5515 */

```

6.3 关键替换点

占位符	替换为
audio_codec_init	CSL_audioClass 调用
dma_audio_start	CSL_dma 配置
intc_enable	CSL_intc 调用
cpu_idle_until_irq	asm(" idle");
函数 audio_isr 加 interrupt 关键字	C55 编译器扩展

7. 配置 Linker .cmd

C5515 工程默认会有一个 lnk_c5515.cmd。需要加入我们的自定义段。

7.1 找到 .cmd 文件

工程根目录下应该有 lnk_c5515.cmd 或 c5515.cmd。如果没有：右键工程 → New → File → 文件名 lnk_c5515.cmd。

7.2 完整 .cmd 模板

```
/* =====
 * lnk_c5515.cmd -- TMS320C5515 linker command file
 * Hearing-aid algorithm port
 * ===== */

-C
-stack 0x1000          /* 4 KB stack -- pipeline state is static */
-heap 0x400            /* 1 KB heap -- minimal, no malloc */

MEMORY
{
    PAGE 0:             /* PROGRAM memory */
        VECT      : origin = 0x000100, length = 0x000100
        ROM       : origin = 0x000200, length = 0x00FE00
        SARAM_P   : origin = 0x010000, length = 0x028000 /* 160 KB */
        DARAM_P   : origin = 0x000000, length = 0x010000 /* 64 KB shared */
        EXT_P     : origin = 0x800000, length = 0x000000 /* unused */

    PAGE 1:             /* DATA memory */
        SARAM_D   : origin = 0x010000, length = 0x028000
        DARAM_D   : origin = 0x000000, length = 0x010000
}

SECTIONS
{
    .vectors      : > VECT      PAGE 0
    .text         : > ROM       PAGE 0
    .cinit        : > ROM       PAGE 0
    .pinit        : > ROM       PAGE 0
    .const        : > ROM       PAGE 0 /* default .const */
    .switch       : > ROM       PAGE 0

    /* Custom sections from our memmap.h: */
    .saram        : > SARAM_D   PAGE 1 /* algorithm state, working bufs */
    .saram.const   : > SARAM_D   PAGE 1 /* large RO tables (hamming, twiddle)
*/
    .daram        : > DARAM_D   PAGE 1 /* FFT scratch, DMA ping-pong */
    .ext          : > SARAM_D   PAGE 1 /* unused; map to SARAM for now */

    .stack        : > DARAM_D   PAGE 1
    .sysmem       : > DARAM_D   PAGE 1
    .bss          : > SARAM_D   PAGE 1
    .data         : > SARAM_D   PAGE 1
    .cio          : > SARAM_D   PAGE 1
}
```

7.3 关键字段含义

段	用途	我们的代码哪些放这
.saram	SARAM 数据段	g_wiener, g_stft, g_vad, pipeline 5 个 working buffers, g_transposed, g_brev
.saram.const	SARAM 只读	hamming_q15[], twiddle_re_q15[], twiddle_im_q15[]
.daram	DARAM 数据	FFT scratch work[], re[], im[], DMA g_dma_in/out
.ext	外部 SDRAM	当前为空

如果你的板子内存映射不一样（不同 EVM 版本可能不同），调 MEMORY 块的 origin 和 length。从 TI 板规格书查。

8. 连接开发板

8.1 物理连接

1. USB 线：一端 PC，一端板上 USB JTAG 口
2. 板上 LED 亮起 = 供电 OK
3. Windows：首次连接会自动装 XDS100v2 驱动（来自 CCS 安装）
4. Linux：检查 lsusb 应看到 Texas Instruments, Inc. XDS100v2 JTAG Probe

8.2 创建 Target Configuration

1. CCS 菜单 View → Target Configurations
2. 在 Target Configurations 视图中右键 User Defined → New Target Configuration
3. 文件名：c5515.ccxml, Use shared location 勾上
4. 打开 .ccxml 编辑器：
 - Connection: Texas Instruments XDS100v2 USB Debug Probe
 - Board or Device: 搜索框输入 "5515" → 选 TMS320C5515
5. 右下角 Save
6. Test Connection（按钮在编辑器右上角附近）：
 - 输出末尾应该看到 All cores connected successfully.
 - 看到 Error connecting → 见 §13 故障排查

8.3 启动 Debug 会话

1. 右键 c5515.ccxml → Launch Selected Configuration
2. CCS 切换到 Debug 透视图（右上角图标）
3. Debug 视图里出现 C55xx_0（CPU 核）
4. 右键 C55xx_0 → Connect Target
5. 状态变为 Connected, CPU 处于 Suspended 状态 = 连接成功

9. 编译 + 加载到板

9.1 编译

1. Project → Build Project (或 Ctrl+B)
2. Console 视图显示编译过程
3. 期望结尾:

```
'Finished building target: hearing_aid.out'
```

4. Problems 视图里 0 errors, warnings 可接受但建议处理

9.2 看构建产物大小

Console 末尾会显示内存占用:

section	length	used	free	addr
SARAM_D	0x28000	0x?	0x?	0x10000
DARAM_D	0x10000	0x?	0x?	0x00000

预期占用 (粗估) :

- .saram.const (窗+twiddle 表) : ~4 KB
- .saram (state + 工作缓冲) : ~30 KB
- .daram (FFT scratch + DMA) : ~12 KB
- .text (代码) : ~30 KB
- 总数: 远小于板上 160+64 KB

9.3 加载到板

1. 在 Debug 视图保持 C55xx_0 Connected
2. Run → Load → Load Program (或 F11)
3. 浏览到 Debug/hearing_aid.out → OK
4. 进度条结束后:
 - Program 加载到 RAM (默认)
 - CPU 暂停在 main() 入口
 - Source 视图自动跳转到 app_main.c 的 main()

9.4 全速运行

1. Run → Resume (F8)
2. CPU 开始执行:
 - pipeline_init() 跑完
 - audio_codec_init(16000) 跑完, AIC3204 配好
 - DMA 启动
 - 进入 idle loop, 等中断

10. 实时音频验证

10.1 接音频线

接口	接到
板上 LINE IN (3.5mm)	手机或 PC 音频输出
板上 LINE OUT (3.5mm)	耳机 或 有源喇叭

10.2 听效果

1. 手机播放音乐或语音
2. 戴上耳机听板子输出
3. 正常人耳听感：
 - 声音"变闷"或"变沉"——高频内容明显被压低
 - 偶尔能听到一些"陌生"的中频细节——那是从高频搬下来的
4. A/B 对比：拔耳机直插手机 vs 插板子，能很明显感受到差别

10.3 Audacity 录板子输出，做频谱对比

1. PC 上装 Audacity（免费）
2. Edit → Preferences → Devices: Recording 选 PC 的 line-in
3. 准备：用音频线把板子 LINE OUT 接 PC 的 LINE IN
4. 同一段音频，先录 5 秒"直通"（不过板子），再录 5 秒"过板子处理"
5. Audacity 里两个 track 都用 Analyze → Plot Spectrum：
 - 直通：5-7 kHz 区有清晰能量（chirp 信号）
 - 过板：5-7 kHz 区下降 5+ dB，2.5-3.5 kHz 区多出对应能量

11. 真实 Cycle 数测量

11.1 用 CCS Clock 工具

1. Run → Clock → Enable（开启 cycle 计数）
2. 在 pipeline_process 入口设断点（app_main.c 里 audio_isr 函数）
3. 在 pipeline_process 调用后设断点
4. Run → Resume（F8）
5. 命中第一个断点时记下 Clock 视图里的 cycles 值
6. Run → Resume 继续
7. 命中第二个断点，cycles 值差就是单次 pipeline_process 的真实周期数

11.2 对比预期

配置	预期 cycles/hop	100 MHz CPU 负载
HWA FFT（本上板）	~15-20K	~1-1.3%
软件 FFT（PC 仿真）	~150K	~9%

